

Guía de ramas, protecciones y commits

Cómo se trabaja con git en el repositorio, para desarrolladores y para los agentes que los asisten: qué está protegido, qué pasos seguir y cómo nombrar ramas y commits.

Pull Request obligatorio

1 aprobación del delegado

CI en verde

Sin push directo

CODEOWNERS

Hotfix

Tags

1. Resumen en una página

Las 7 reglas

1. **Una rama por tarea.** Nunca se trabaja sobre `main` ni `develop`.
2. **Prohibido el push directo** a `main` y `develop`. Todo entra por Pull Request.
3. **1 aprobación** del delegado (code owner) del área que se toca.
4. **La CI tiene que pasar** en verde para poder mergear.
5. Las ramas nacen de `develop`. Solo `hotfix/` nace de `main`.
6. El PR apunta a `develop`, salvo release y hotfix (van a `main`).
7. Cada versión en `main` lleva un **tag** `vX.Y.Z`.

Prefijos válidos

Ramas `<prefijo>/<área>-<descripción>`

`feat/` `fix/` `chore/` `docs/` `test/` `refactor/`
`hotfix/`

Commits `tipo: descripción`

`feat` `fix` `chore` `docs` `test` `refactor` `release`

Áreas `back-` · `front-` · *sin área* si toca ambas, infra o docs.

Ejemplo completo:

`feat/back-registro-usuario`

`feat: endpoint de registro de usuario`

Flujo normal, de punta a punta

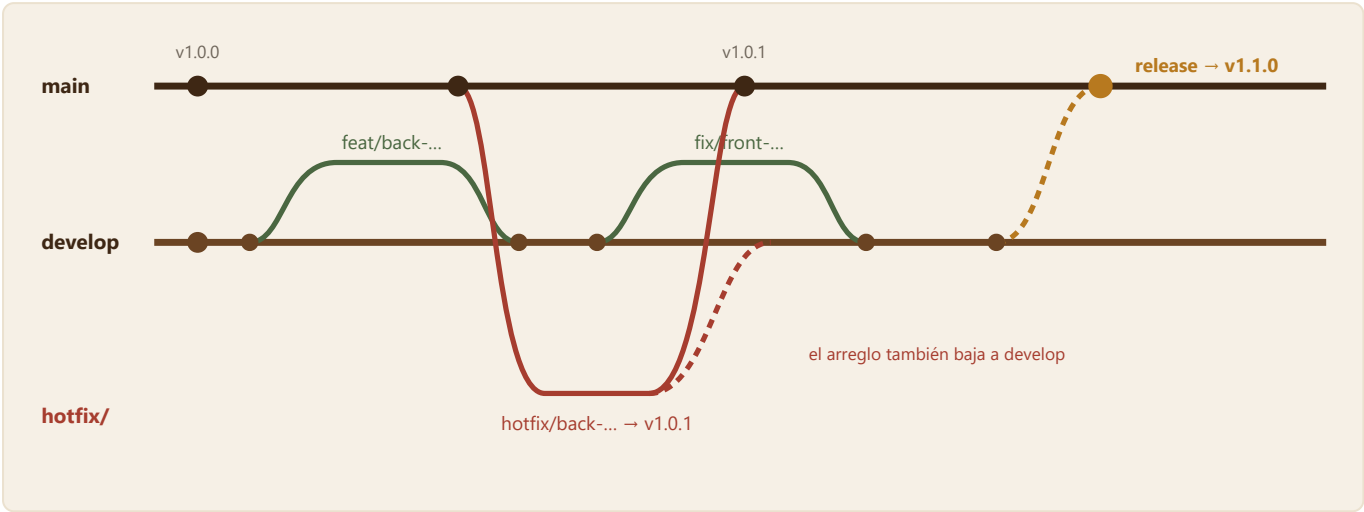


SI ALGO TE LO IMPIDE, NO LO RODEES

Si GitHub rechaza un push, bloquea un merge o la CI falla, **se arregla la causa**. No se fuerza, no se usa `--no-verify`, no se pide a un administrador que saltee la protección. Las protecciones existen para que cuatro personas (y sus agentes) trabajen en paralelo sin pisarse.

2. Mapa de ramas

Hay solo **dos ramas permanentes**. Todo lo demás es temporal y se borra al mergear.



Rama	Tipo	Qué contiene	Cómo entra código
main	Permanente	Lo que corre en producción. Cada merge lleva un tag de versión.	PR desde develop (release) o desde hotfix/
develop	Permanente	Integración: todo lo terminado y revisado, listo para el próximo release.	PR desde las ramas de tarea
feat/ fix/ chore/ ...	Temporal	Una sola tarea.	Se crean desde develop
hotfix/	Temporal	Un arreglo urgente en producción.	Se crea desde main

NO HAY RAMAS BACK NI FRONT

El repo es un monorepo y muchas funcionalidades tocan las dos partes. El área se identifica por el nombre de la rama y la revisión la reparte **CODEOWNERS** (§6).

3. Prefijos de ramas y de commits

3.1 Ramas

Formato: <prefijo>/<área>--<descripción>

Prefijo	Para qué	Ejemplo
feat/	Funcionalidad nueva	feat/back-registro-usuario
fix/	Corrección de un bug (no urgente)	fix/back-webhook-idempotencia
chore/	Configuración, build, dependencias	chore/flyway-migraciones
docs/	Documentación	docs/decisiones-arquitectura
test/	Tests	test/back-inventario-service
refactor/	Reordenar código sin cambiar lo que hace	refactor/front-carrito
hotfix/	Arreglo urgente en producción (§7)	hotfix/back-webhook-mp

El área

La tarea toca	Área	Ejemplo
Solo tierra-backend/	back-	feat/back-registro-usuario
Solo tierra-frontend/	front-	feat/front-tema-oscuro
Las dos, infra o docs	(sin área)	feat/login, chore/flyway-migraciones

Opcional: ID de tablero tras el área (feat/back-H2-registro-usuario). Reglas: minúsculas, guiones, sin espacios ni tildes.

3.2 Commits

Formato: tipo: descripción en presente, en minúscula

Tipo	Cuándo	Ejemplo
feat	Funcionalidad nueva	feat: agregar endpoint GET /api/categorias
fix	Corrección de un bug	fix: incrementar usos_actuales al aplicar un cupon
chore	Configuración, dependencias, build	chore: actualizar dependencia de flyway
docs	Documentación	docs: documentar el flujo de consulta de alquiler
test	Tests	test: cubrir liberacion de reservas vencidas
refactor	Reordenar sin cambiar comportamiento	refactor: extraer el calculo de descuento a un metodo propio
release	Título del PR develop → main	release: v1.1.0

- El prefijo del commit coincide con el de la rama (feat/... → feat:). Un hotfix/ usa fix: .
- Un commit se explica en **una línea**. Si necesitas una "y", probablemente son dos commits.
- El **título del PR** sigue este formato: con *Squash and merge* pasa a ser el commit en develop .
- No hace falta que cada commit deje la app andando (para eso está la rama), pero sí lo que llega a develop y main .

4. Pasos al trabajar en una tarea

1 Actualizá `develop` y partí de ahí.

```
git switch develop
git pull
```

2 Creá la rama de la tarea (§3).

```
git switch -c feat/back-registro-usuario
```

3 Verificá la rama antes de editar.

```
git branch --show-current
```

4 Trabajá y commiteá (formato §3).

```
git add <archivos>
git commit -m "feat: registro de usuario"
```

5 Traé lo último de `develop`.

```
git pull --rebase origin develop
```

6 Subí la rama.

```
git push -u origin feat/back-registro-usuario
```

7 Abrí el PR hacia `develop`. Base: `develop`. Título con formato; describí qué cambia y cómo probarlo.

8 Esperá CI en verde y aprobación del delegado. Resolvé comentarios. Commits nuevos descartan la aprobación.

9 Squash and merge. Luego actualizá tu `develop` local:

```
git switch develop
git pull
```

EL PR APUNTA A DEVELOP, NO A MAIN

Un PR de tarea contra `main` salta la integración. Solo `release` y `hotfix` van a `main`.

DOS COMANDOS QUE SALVAN EL DÍA

- **Editaste sin rama:** `git switch -c feat/xxx` se lleva los cambios.
- **Cambiar de rama a medias:** `git stash` y `git stash pop`.

Checklist antes de abrir el PR

- ☐ Backend compila y arranca (`.\mvnw spring-boot:run`)
- ☐ Frontend compila (`npm run build`)
- ☐ `git status` limpio de archivos no deseados
- ☐ Sin credenciales, tokens ni rutas locales en diff
- ☐ Flyway: migración nueva y entidades actualizadas
- ☐ Rebase con `develop` sin conflictos
- ☐ PR apunta a `develop` (o `main` si `release/hotfix`)
- ☐ Rama y PR siguen los prefijos de la §3

5. Protección de ramas

Se configura una sola vez en GitHub (*Settings* → *Branches*, o como *Ruleset*) y lo hace quien administra el repositorio. Se aplica igual a `main` y a `develop`.

Ajuste	<code>main</code>	<code>develop</code>	Qué consigue
Require a pull request before merging	✓	✓	Bloquea el push directo. Nada entra sin PR.
Required approvals	1	1	Una aprobación humana como mínimo.
Require review from Code Owners	✓	✓	Esa aprobación tiene que ser del delegado del área (§6).
Dismiss stale approvals on new commits	✓	✓	Si cambia el código después de aprobar, hay que re-aprobar.
Require status checks to pass (CI)	✓	✓	No se mergea con la CI en rojo. Ver nota.
Require branches to be up to date	✓	✓	El PR se probó contra lo último de la rama destino.
Require conversation resolution	✓	✓	No quedan comentarios de revisión sin resolver.
Include administrators / sin bypass	✓	✓	Las reglas valen también para quien administra.
Allow force pushes	✗	✗	No se reescribe el historial compartido.
Allow deletions	✗	✗	Las ramas permanentes no se borran.

Ajustes generales del repositorio

- **Default branch:** `develop`, para que los PRs nuevos apunten ahí por defecto.
- **Automatically delete head branches:** activado, para que las ramas de tarea se borren solas al mergear.
- **Tags v* protegidos** (ruleset de tags): una vez creado, un tag no se mueve ni se borra.

Cómo queda en la práctica

Bloqueado

- `git push origin main / develop`
- `git push --force` sobre ramas compartidas
- Mergear sin aprobación del delegado
- Mergear con la CI en rojo o sin correr
- Mergear el propio PR sin otra persona que apruebe

Permitido

- Pushear a tu rama de tarea (`feat/...`, `fix/...`)
- Reescribir *tu* rama con `--force-with-lease` tras un rebase (solo si nadie más la usa)
- Abrir PRs en borrador para pedir opinión temprana

ESTADO DE IMPLEMENTACIÓN

- **CI:** todavía no hay workflows en `.github/` (llegan en la fase 5). Cuando existan, se agrega *Require status checks* con los nombres de sus jobs (compilación y tests de backend, build de frontend). Hasta entonces, el control es el checklist de la §4 y los scripts `probar-*.ps1`.
- **CODEOWNERS:** el archivo existe pero sus líneas están comentadas; faltan los usuarios reales de GitHub (§6).
- **develop:** se crea a partir de `main` cuando se mergea la documentación del flujo.

6. Delegado y CODEOWNERS

El **delegado** es la persona responsable de revisar y aprobar los cambios de un área del proyecto. En GitHub se formaliza con `.github/CODEOWNERS`: cuando un PR toca esas carpetas, GitHub le pide la revisión automáticamente, y con *Require review from Code Owners* el PR **no se puede mergear sin su aprobación**.

```
# .github/CODEOWNERS – si varias líneas coinciden, vale la última
/tierra-backend/                @delegado-back
/tierra-frontend/              @delegado-front
/tierra-infra/                 @administrador
/tierra-backend/src/main/resources/db/migration/ @delegado-back @administrador
/.github/                      @administrador
/CONTRIBUTING.md              @administrador
/docs/decisiones/             @administrador
```

Si el PR toca...	Aprueba
<code>tierra-backend/</code>	Delegado de backend
<code>tierra-frontend/</code>	Delegado de frontend
Migraciones Flyway	Delegado de backend y administrador
<code>tierra-infra/</code> , <code>.github/</code> , <code>CONTRIBUTING.md</code> , <code>docs/decisiones/</code>	Administrador
Backend y frontend a la vez	Ambos delegados

Reglas de la aprobación

- Se exige **al menos 1 aprobación**, y tiene que ser de un code owner del área tocada.
- **Nadie aprueba su propio PR**. Si el autor es el delegado, aprueba el otro delegado o el administrador.
- Los usuarios del archivo deben existir y tener **permiso de escritura** en el repo; si no, GitHub marca la regla como inválida y deja de asignar revisores.
- Los cambios al propio `CODEOWNERS` van por PR como cualquier otro, y los aprueba el administrador.
- Un agente puede *preparar* un PR, pero la aprobación es siempre de una persona.

7. Hotfix: arreglos urgentes en producción

Se usa **solo** cuando hay un bug grave en lo que ya está en producción y no se puede esperar al ciclo normal. Es la única rama que nace de **main**: ahí está lo que corre en producción, mientras que **develop** puede tener trabajo a medio validar.

¿ES UN HOTFIX?

Sí: la producción está caída, se pierden pedidos o pagos, hay una falla de seguridad. **No:** un bug molesto pero tolerable, algo que se puede planificar → **fix/** desde **develop**.

- 1 Partí de **main** actualizada y creé la rama.

```
git switch main
git pull
git switch -c hotfix/back-webhook-mp
```

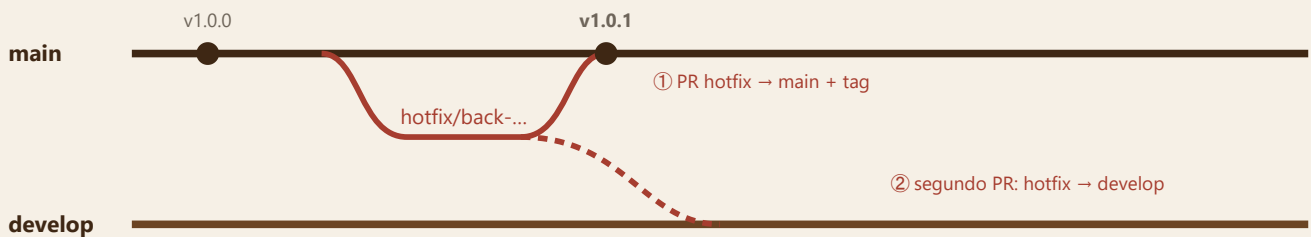
- 2 Arreglá **SOLO** ese problema. Nada de mejoras ni limpiezas de paso. Commits con **fix:**.

- 3 Subí y abrí PR **hotfix/...** → **main**. Mismas protecciones: 1 aprobación del delegado y CI en verde. Se mergea con *Create a merge commit*.

```
git push -u origin hotfix/back-webhook-mp
```

- 4 Publicá el tag de **PARCHE** sobre el commit resultante en **main** (ver §8): **v1.0.0** → **v1.0.1**.

- 5 Abrí un segundo PR **hotfix/...** → **develop** para que el arreglo no se pierda en el próximo release. **Es el paso que más se olvida.** Merge commit también.



8. Releases y tags

Una release es el momento en que lo que hay en `develop` pasa a producción.

- 1 **Abrí un PR `develop` → `main`** con título `release: vX.Y.Z` y la lista de lo que incluye.
- 2 **Revisión y merge** con *Create a merge commit*. Mismas protecciones: delegado, CI en verde.
- 3 **Poné el tag** al commit resultante en `main` y publicalo:

```
git switch main
git pull
git tag -a v0.1.0 -m "Primera version desplegable"
git push origin v0.1.0
```

- 4 **(Opcional)** Creá la *Release* en GitHub desde el tag, con el listado de cambios.

QUÉ ES UN TAG

Una etiqueta fija sobre un commit: a diferencia de una rama, **no se mueve**. Sirve para saber qué código corría en cada momento, volver a una versión anterior si la nueva falla y ubicar en qué versión apareció un bug.

Versionado semántico: `vMAYOR.MENOR.PARCHE`

Qué cambió	Qué número sube	Ejemplo
Un bug corregido (típicamente un <code>hotfix/</code>)	PARCHE	<code>v1.0.0</code> → <code>v1.0.1</code>
Funcionalidad nueva que no rompe nada	MENOR	<code>v1.0.1</code> → <code>v1.1.0</code>
Un cambio que rompe la compatibilidad	MAYOR	<code>v1.1.0</code> → <code>v2.0.0</code>

Mientras el proyecto no esté en producción se usa `v0.x.y`. La `v1.0.0` es la primera versión pública.

Estrategia de merge según el PR

PR	Cómo se mergea en GitHub	Por qué
Rama de tarea → <code>develop</code>	Squash and merge	Un commit por tarea; <code>develop</code> queda legible, sin los "wip".
<code>develop</code> → <code>main</code> (release)	Create a merge commit	Conserva el historial y mantiene <code>main</code> y <code>develop</code> alineadas.
<code>hotfix/</code> → <code>main</code> y → <code>develop</code>	Create a merge commit	Mismo motivo.

NUNCA SQUASH DE DEVELOP A MAIN

Git deja de reconocer que `develop` ya está incluida en `main`: las ramas divergen y el release siguiente aparece con conflictos que nadie provocó.

9. Reglas para agentes de IA

Los agentes que trabajan en el repo (asistentes de código, automatizaciones) siguen **las mismas reglas** que las personas, más las siguientes. Un agente actúa con las credenciales de quien lo lanza: lo que haga queda a nombre de esa persona, y esa persona es responsable del resultado.

Siempre

- Verificar la rama con `git branch --show-current` **antes de editar**.
- Crear una rama de tarea desde `develop` actualizada, con prefijo y área (\$3).
- Commitear con formato `tipo: descripción`, de a un cambio lógico.
- Traer `develop` con `git pull --rebase origin develop` antes de subir.
- Abrir PR con base `develop`, explicando qué cambia y cómo probarlo.
- Revisar el diff antes de commitear: sin secretos, sin `.env`, sin rutas locales.
- Mantener el alcance: **una tarea = una rama = un PR**.
- Si algo está bloqueado o falla, **parar e informar** a la persona.

Nunca

- Commitear o pushear sobre `main` o `develop`.
- `git push --force` a ramas compartidas; `reset --hard` sobre trabajo ajeno.
- Usar `--no-verify` ni saltar hooks o checks.
- Aprobar o mergear su propio PR, ni pedir bypass de las protecciones.
- Crear, mover o borrar **tags** ni publicar releases (lo hace el delegado/administrador).
- Abrir un `hotfix/` sin que una persona lo declare urgente.
- Editar una migración Flyway ya mergeada (se crea una nueva).
- Mezclar tareas distintas en una misma rama o PR.
- Escribir tokens, passwords o claves en ningún archivo.

Secuencia mínima de un agente

```
# 1. Punto de partida limpio
git status                    # sin cambios ajenos pendientes
git switch develop; git pull
git switch -c fix/back-webhook-idempotencia

# 2. Cambio + commit
git add <archivos concretos> # nunca "git add ." a ciegas
git commit -m "fix: ignorar webhook duplicado de mercado pago"

# 3. Integrar y subir
git pull --rebase origin develop
git push -u origin fix/back-webhook-idempotencia

# 4. Abrir PR (base: develop) y detenerse a esperar CI + aprobación del delegado
```

ANTE LA DUDA, PREGUNTAR

Si la tarea toca los mismos archivos que la rama de otra persona, o no está claro si es `fix/` o `hotfix/`, el agente **consulta antes de seguir**. Coordinarse cinco minutos es más barato que resolver un conflicto después.

10. Referencia rápida

Quiero...	Hago
Empezar una tarea	<code>git switch develop && git pull && git switch -c feat/back-xxx</code>
Saber en qué rama estoy	<code>git branch --show-current</code>
Traer lo último antes de subir	<code>git pull --rebase origin develop</code>
Subir mi rama por primera vez	<code>git push -u origin <mi-rama></code>
Rescatar cambios hechos sin rama	<code>git switch -c feat/lo-que-sea</code>
Guardar cosas a medias	<code>git stash / git stash pop</code>
Arreglar un bug en producción	<code>git switch main && git pull && git switch -c hotfix/back-xxx</code>
Etiquetar una versión	<code>git tag -a vX.Y.Z -m "..." && git push origin vX.Y.Z</code>
Después de mergear mi PR	<code>git switch develop && git pull</code>

Mensajes de error frecuentes

Lo que ves	Qué pasó
<code>push rejected ... protected branch</code>	Intentaste pushear a <code>main</code> o <code>develop</code> . Creá una rama y abrí un PR.
<code>rejected ... (fast-forward)</code> al subir tu rama	Creaste la rama desde un <code>develop</code> viejo. <code>git pull --rebase origin develop</code> y volvé a subir.
"Review required" / merge bloqueado	Falta la aprobación del code owner del área, o se descartó por commits nuevos.
"Required status check is expected"	La CI no corrió o falló. Revisá el log del job y arreglá la causa.
"Branch is out-of-date"	<code>develop</code> avanzó. Actualizá la rama del PR (rebase) y esperá la CI otra vez.

Toda la guía en una imagen

Cada cambio, sin excepciones

1. Rama nueva desde **develop** (o desde **main** si es hotfix)
2. Nombre **prefijo/área-descripción** · commits **tipo: descripción**
3. **Pull Request** — nunca push directo a main ni develop
4. **CI en verde** + **1 aprobación del delegado** (CODEOWNERS)
5. Squash a develop · merge commit a main · **tag vX.Y.Z** en cada release o hotfix
6. Hotfix: PR a main y segundo PR a develop

Fuente de verdad: **CONTRIBUTING.md** §4 (ramas y flujo) y §5 (commits), [.github/CODEOWNERS](#). Este documento es un resumen operativo para consulta rápida; si cambia el flujo, se actualiza primero **CONTRIBUTING.md** y luego se regenera este PDF.